

A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems

Jin Xie, Xinyu Li, Liang Gao^{*}, Lin Gui

State Key Laboratory of Digital Manufacturing Equipment and Technology, Huazhong University of Science and Technology, Wuhan 430074, China

ARTICLE INFO

Keywords:

Distributed and flexible job shop scheduling
Genetic algorithm
Tabu search
Neighborhood structure

ABSTRACT

The distributed flexible job-shop scheduling problem (DFJSP) is an extension of the flexible job shop scheduling problem, which is a famous NP-complete combinatorial optimization problem. DFJSP consists of three sub-problems, assigning jobs to appropriate factories, scheduling operations to suitable machines, and determining the operation sequence on machines. Due to the complexity of DFJSP, there is only a little research on the problem. There are several benchmark sets proposed to test the performance of methods for solving DFJSP. However, many instances remain unresolved after a long time. This paper proposes a hybrid genetic tabu search algorithm (HGTSA) for DFJSP. The proposed HGTSA combines the global search ability of the genetic algorithm (GA) and the local search ability of the tabu search (TS) well. Two genetic operators are designed based on the critical factory to discrete the population. A new neighborhood structure is introduced into TS to search for more space of neighborhood solutions. To evaluate the performance of HGTSA, this paper compares it with four state-of-the-art algorithms on 69 benchmark instances. The experimental results demonstrate that HGTSA outperforms these comparison algorithms in terms of solution quality and computation efficiency. In particular, HGTSA has found 13 newly upper bounds of these benchmark instances.

1. Introduction

Shop scheduling is one of the most significant issues in the planning and managing of production manufacturing. There are many styles of shop scheduling problems, such as flow shop scheduling problems [1–3], job shop scheduling problems [4,5], and flexible job-shop scheduling problems [6,7]. Among these scheduling problems, flexible job-shop scheduling problems (FJSP) are regarded as the most difficult because it needs to consider both selecting appropriate machines for operations and determining the operation sequence on machines. In recent years, many researchers have focused a lot of attention on FJSP because it is widely applied to the field of industrial manufacturing. For example, Zhang et al. [8] proposed an improved genetic algorithm (GA) to solve FJSP. A crossover operator with artificial pairing was designed to improve poor solutions and an adaptive mutation mechanism was devised to help individuals jump out of the local optimum. Zhu and Zhou [9] presented a grey wolf optimizer for solving FJSP. Their algorithm introduced an improved social hierarchy to increase the convergence speed and developed a diverse leader strategy to enhance the population diversity. Gong et al. [10] designed an artificial bee colony algorithm to

solve FJSP. An effective local search method is employed to improve the searching speed and the exploitation ability. Fan et al. presented a hybrid Jaya algorithm for solving FJSP. A tabu search (TS) procedure is incorporated into their algorithm to enhance its local search ability.

Analyzing the above literature, we observe that most of the existing research on FJSP focuses on a single factory. However, with the globalization and collaborative production of multi-factory enterprises, distributed manufacturing becomes more and more competitive. It brings in many advantages such as production flexibility, low production costs, and low management risks [11]. Compared with single-factory scheduling, there are relatively few papers on distributed scheduling. Naderi and Ruiz [12] first gave the definition of distributed flow shop scheduling problems (DPFSP), and formulated six different mixed integer linear programming models for DPFSP. Lu et al. [13] proposed a pareto-based collaborative multi-objective optimization algorithm for DPFSP. A speed scaling strategy based on problem property is designed to reduce the total energy consumption and a collaborative initialization strategy is presented to generate a high-quality initial population. Mao et al. [14] devised an iterated greedy algorithm for DPFSP. A destruction phase with a tournament selection and a construction phase with an enhanced strategy was introduced to avoid the

^{*} Corresponding author.

E-mail address: gaoliang@mail.hust.edu.cn (L. Gao).

<https://doi.org/10.1016/j.jmsy.2023.09.002>

Received 25 July 2023; Received in revised form 2 September 2023; Accepted 3 September 2023

Available online 9 September 2023

0278-6125/© 2023 The Society of Manufacturing Engineers. Published by Elsevier Ltd. All rights reserved.

Nomenclature

i, i'	indexes for jobs
n	total number of jobs
I	set of jobs, where $I = \{1, 2, \dots, n\}$
j, j'	index of operations
n_i	operation number of job i
J_i	operation set of job i , where $J_i = \{1, 2, \dots, n_i\}$
JJ_i	partial operation set of job i , where $J_i = \{1, 2, \dots, n_i-1\}$
k, k'	indexes of machines
O_{ij}	the j th operation of job i
f	index of factory
r_f	number of factories
F	set of factories, where $F = \{1, \dots, r_f\}$

m_f	number of machines in factory f
m_{ijf}	number of machines eligible for processing O_{ij} in factory f
K_f	machine set in factory f , where $K_f = \{1, 2, \dots, m_f\}$
K_{ijf}	set of machines eligible for processing O_{ij} in factory f
pt_{ijk}	processing time of operation O_{ij} processed on machine k in factory f
M	a large positive number
X_{ijk}	if operation O_{ij} is assigned on machine k in factory f , $X_{ijk} = 1$; otherwise, $X_{ijk} = 0$
$Y_{iji'k'}$	if operation O_{ij} is processed before operation $O_{i'j'}$, $Y_{iji'k'} = 1$; otherwise $Y_{iji'k'} = 0$
Z_{if}	if job i is assigned to factory f , $Z_{if} = 1$; otherwise $Z_{if} = 0$
B_{ij}	start time of operation O_{ij}
$C_{\max}$	makespan

local optimum. Jia et al. [15] first studied the distributed job shop scheduling problem (DJSP) and adopted a GA with once gene crossover and twice gene mutation to solve it. Sang et al. [16] used a collaborative scheduling method for many-objective distributed flexible job shop in a intelligent factory. A many-objective distributed flexible job shop collaborative scheduling model was established and a high-dimensional many-objective memetic algorithm was proposed for it. Şahman [17] developed a discrete spotted hyena optimizer for DJSP. A workload-based device order mechanism was integrated into the algorithm to allocate jobs to machines reasonably, and 80 distributed job shop scheduling problems were solved by the algorithm. From the above literature, we can find that the study of distributed shop scheduling problem is of great significance, which can effectively improve production flexibility and reduce production costs.

However, most distributed shop scheduling problems mainly focuses on DPFSP [18–20], and rarely on distributed and flexible job shop scheduling problems (DFJSP), which can adapt to small-scale customized production and geographically dispersed production well. Therefore, the research on DFJSP is very critical and urgent. Giovanni and Pezzella [21] first defined the concept of DFJSP and designed an improved GA to solve the problem. Besides traditional crossover and mutation operators, a local search-based operator was used to refine the most promising individual of each generation. Luo et al. [22] proposed an efficient memetic algorithm for solving DFJSP. Various crossover and mutation operators and three types of neighborhood structures were designed to expand the solution space and accelerate the convergence speed. Zhu et al. [23] solved a dynamic distributed flexible job-shop scheduling problem with operation inspection. A modified memetic algorithm with a novel encoding method and an initialization method was designed for it. Lu et al. [24] developed a GA embedded with a concise chromosome representation for DFJSP. A 1-dimensional encoding and 3-dimensional decoding method were convenient to generate feasible solutions. By analyzing the above papers, we find that meta-heuristics are the most effective approaches for DFJSP. Hence, this paper also design a powerful meta-heuristic to solve DFJSP.

GA is one of the most famous and effective swarm intelligence optimization algorithms. If the crossover and mutation operators are well designed, GA can have a strong global search ability and easily jump out of the local optimum. TS has a powerful local search ability. By using a tabu list, TS can avoid re-visiting the solutions which have been searched previously. Many researchers have combined GA and TS for solving the shop scheduling problems and achieved excellent results. For instance, Palacios et al. [25] proposed a hybrid algorithm that combines GA and TS to solve FJSP. The TS was applied to each chromosome in GA to search for neighborhood solutions. Zhang et al. [26] mixed GA with TS to solve the job shop scheduling problem. The genetic operators and the TS procedure struck a balance of exploration and exploitation well. Li and Gao [27] combined GA and TS for solving FJSP. Their hybrid

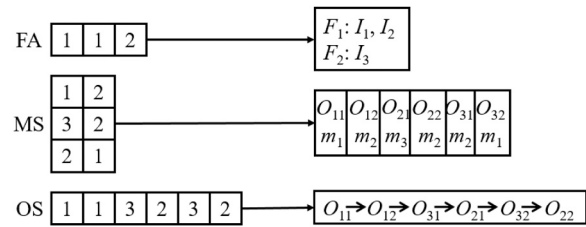


Fig. 1. Schematic diagram of the encoding scheme.

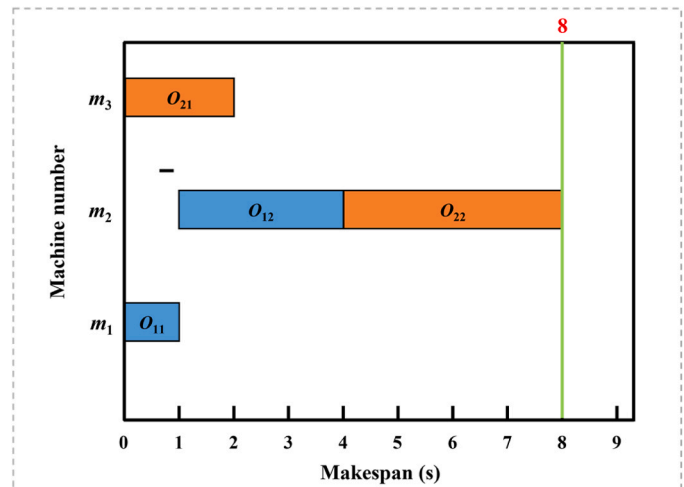


Fig. 2. The Gantt graph of F_1 .

Table 1

A case of DFJSP with three jobs and two factories.

Job	Operation	F_1			F_2		
		m_1	m_2	m_3	m_1	m_2	m_3
I_1	O_{11}	1	-	2	4	-	2
	O_{12}	2	3	-	2	-	3
I_2	O_{21}	3	-	1	-	3	-
	O_{22}	2	4	-	-	4	3
I_3	O_{31}	-	1	-	-	2	4
	O_{32}	3	5	-	2	3	-

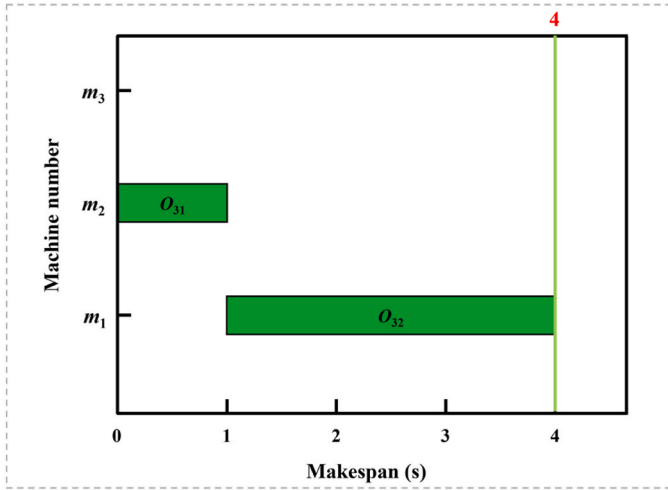
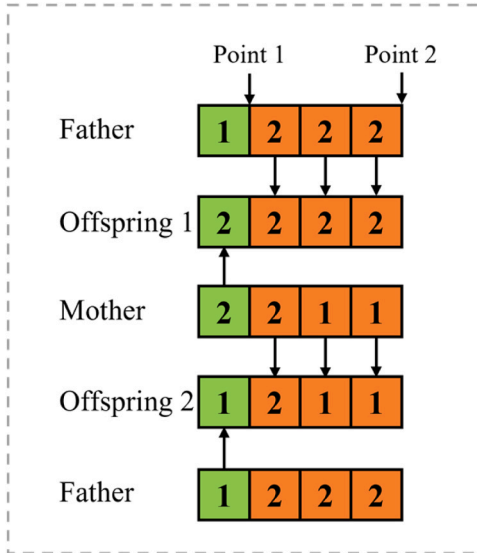
Fig. 3. The Gantt graph of F_2 .

Fig. 4. The schematic example of factory crossover.

algorithm has updated many upper bounds for several benchmark instances. Considering the effectiveness of combining GA and TS to solve shop scheduling problems, this paper also designs a hybrid genetic tabu search algorithm (HGTSA) for DFJSP and compares it with four state-of-the-art algorithms for 69 benchmark instances. In particular, HGTSA has updated 13 upper bounds out of these difficult instances.

The remainder of this paper is organized as follows. The problem description and its integer programming model of DFJSP are introduced in Section 2. The algorithm framework of HGTSA is described in Section 3. The experimental results of HGTSA on the DFJSP benchmark are

shown in Section 4. Finally, the conclusion and future works are provided in Section 5.

2. Problem formulation

The DFJSP can be described as follows: a set of jobs $I = \{1, 2, \dots, n\}$ has to be processed in a set of factories $F = \{1, \dots, r_f\}$. Each factory contains a flexible job shop scheduling problem. Each job i consists of a list of operations $O = (O_{i1}, O_{i2}, \dots, O_{in_i})$ and has its own processing route. Each operation O_{ij} has to be processed on a predetermined subset of candidate machines in a factory $K_{ijf} \subseteq K_f$. It is important to note that all operations of the same job have to be processed in the same factory. Besides, this paper assumes that all factories, jobs, and machines are available at time zero. In each factory, each machine can process at most one operation at a time. Each operation can be assigned to only one factory, and scheduled to only one machine in the factory. Once a processing process of operations starts, it cannot be interrupted until the processing is finished. The objective of DFJSP is to minimize the global makespan of all factories. A detailed mathematical formulation is directly taken from Meng et al. [28].

The mathematical model of DFJSP is given as follows:

$$\sum_{f \in F} Z_{ijf} = 1, \quad \forall i \in I \quad (1)$$

$$Z_{ijf} = \sum_{k \in K_{ijf}} X_{ijk}, \quad \forall i \in I, j \in J_i, f \in F \quad (2)$$

$$B_{ij} + \sum_{f \in F} \sum_{k \in K_{ijf}} p_{t_{ijk}} X_{ijk} \leq B_{ij+1}, \quad \forall i \in I, j \in J_i \quad (3)$$

$$B_{ij} + p_{t_{ijk}} X_{ijk} \leq B_{i'j'} + M(3 - Y_{ijf} - X_{i'j'k} - Y_{i'j'k}) \\ \forall i, i' \in I, i < i', j \in J_i, j' \in J_{i'}, \\ f \in F, k \in K_{ijf} \cap K_{i'j'k} \quad (4)$$

$$B_{ij} + p_{t_{ijk}} X_{ijk} \leq B_{ij} + M(2 + Y_{ijf} - X_{ijk} - Y_{i'j'k}) \\ \forall i, i' \in I, i < i', j \in J_i, j' \in J_{i'}, \\ f \in F, k \in K_{ijf} \cap K_{i'j'k} \quad (5)$$

$$C_{\max} \geq B_{in_i} + \sum_{f \in F} \sum_{k \in K_{in_if}} (p_{t_{in_ifk}} X_{in_ifk}), \quad \forall i \in I \quad (6)$$

$$B_{ij} \geq 0 \quad \forall i \in I, j \in J_i \quad (7)$$

where constraint (1) guarantees each job should be assigned to only one factory. Constraint (2) ensures all operations of a job have to be processed in the same factory. Besides, constraint (2) guarantees each operation can be scheduled on only one machine. Constraint (3) restricts the operations of a job to be processed in a pre-determined order. Constraint (4) – (5) ensures the non-overlap of operations on the same machine. Constraint (6) calculates the makespan. Constraint (7) guarantees the decision variables are non-negative.

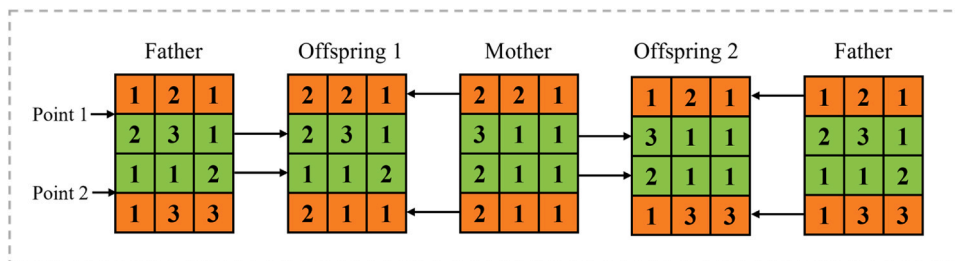


Fig. 5. The schematic example of machine crossover.

3. The HGTS algorithm

3.1. Main framework

The hybrid genetic tabu search algorithm (HGTS) combines the genetic algorithm (GA) and the tabu search (TS) algorithm for solving DFJSP. Because GA has powerful global search ability and TS has strong local search ability, HGTS balances the exploration and the exploitation well. The main procedure of HGTS is presented in Algorithm 1. The details of HGTS will be introduced in the following sub-sections. The overall procedure of HGTS is provided as follows:

Step 1: Parameter settings. Set population size PS , crossover probability P_c , mutation probability P_m , and maximum iterations of TS $maxIter$.

Step 2: Population initialization. Randomly generate an initial population P .

Step 3: Fitness evaluation. Calculate the fitness value of individuals in P by the objective function.

Step 4: Crossover operator. Use the roulette wheel to choose father x_f and mother x_m . Perform a crossover operator for x_f and x_m to generate two offspring individuals.

Step 5: Mutation operator. Perform a mutation operator for individuals in P to generate new individuals.

Step 6: Tabu search algorithm. For each individual in P , perform a local search procedure to search its neighborhood solutions.

Step 7: Termination condition. If the makespan of x_b reaches the lower bound or the total iteration number exceeds $maxIter$, output the optimal individual x_b ; otherwise, continue to Step 4.

The critical elements of the proposed HGTS include an encoding scheme, a decoding method, a crossover operator, a mutation operator, and a tabu search algorithm, which are explained in detail below.

Algorithm 1. The main framework of HGTS.

```

1: Input: factory set  $F$ , job set  $I$ , machine set  $K_f$ , and processing time  $pt_{ijk}$  ( $\forall i, j, f, k$ )
2: Output: the best solution  $x_b$  found so far
3: Set the parameters of HGASA
4: Randomly initialize a population  $P = \{x_1, x_2, \dots, x_p\}$ , and  $x_b \leftarrow$  select the best solution among  $P$ 
5: While (the termination condition is satisfied) do
6:    $Q \leftarrow \emptyset$ 
7:   // crossover operator
8:   For  $i = \{1, 2, \dots, p/2\}$  do
9:     Use the roulette wheel to choose two parents  $x_f$  and  $x_m$  from  $P$ 
10:    Randomly generate a decimal  $r_c \in [0, 1]$ 
11:    If  $r_c < p_c$  then //  $p_c$  is the crossover probability
12:       $Q = Q \cup \text{Crossover}(x_f, x_m)$ 
13:    Else
14:       $Q = Q \cup (x_f, x_m)$ 
15:    End if
16:  End for
17:  // mutation operator
18:  For  $i = \{1, 2, \dots, p\}$  do
19:    Randomly generate a decimal  $r_m \in [0, 1]$ 
20:    If  $r_m < p_m$  then //  $p_m$  is the mutation probability
21:       $y_i \leftarrow \text{Mutation}(y_i)$  //  $Q = \{y_1, y_2, \dots, y_p\}$ 
22:    End if
23:  End for
24:  // tabu search procedure
25:  For  $i = \{1, 2, \dots, p\}$  do
26:     $y_j \leftarrow \text{TabuSearch}(y_i)$ 
27:  End for
28:   $P \leftarrow Q$ 
29:   $x_b' \leftarrow$  select the best solution among  $P$ 
30:  If  $C(x_b') < C(x_b)$  then
31:     $x_b \leftarrow x_b'$ 
32:  End if
33: End while
34: Return  $x_b$ 

```

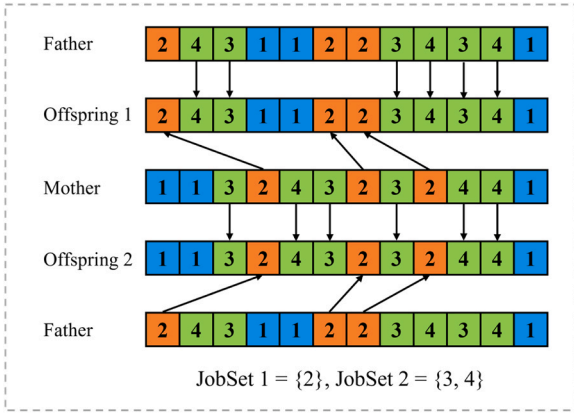


Fig. 6. The schematic example of operation crossover.

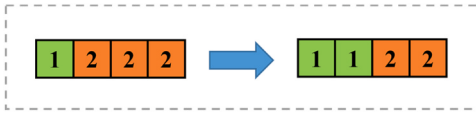


Fig. 7. The schematic example of factory mutation.

3.2. Encoding scheme

An appropriate encoding scheme is crucial for solving DFJSP. Different from the traditional FJSP, DFJSP needs to consider not only scheduling operations to suitable machines and determining operation sequence on machines but also assigning jobs to appropriate factories. Besides, an exemplary encoding scheme should facilitate executing crossover, mutation, and tabu search operators. Also, the feasibility of solutions should be guaranteed after executing these operators. This paper designs a three-layer encoding scheme including factory assignment (FA), machine scheduling (MS), and operation sequence (OS) to represent a scheduling solution. FA denotes the assignment of jobs to machines, MS represents the scheduling of operations on machines, and OS describes the processing sequence of operations on machines.

Table 1 provides a case of DFJSP to explain our encoding scheme in detail. Three jobs have to be assigned to two factories first. Each job consists of two operations. Then, the processing machine and the processing route of each operation should be determined. Fig. 1 gives a schematic diagram of the coding scheme corresponding to the case. FA presents that jobs I_1 and I_2 are assigned to factory F_1 , job I_3 is assigned to factory F_2 . In MS, the element in the i -th row and the j -th column represents the processing machine of operation O_{ij} . For example, the element in the first row and the second column is 2. It represents that the processing machine of operation O_{ij} is machine m_2 . OS denotes the processing sequence of operations. The number of times the job appears is its corresponding operation number. Hence, operation O_{11} is first scheduled and then operation O_{12} is scheduled. The remaining operations are scheduled according to OS in turn.

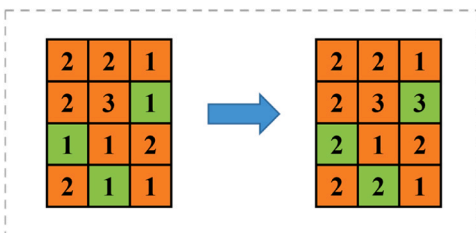


Fig. 8. The schematic example of machine mutation.

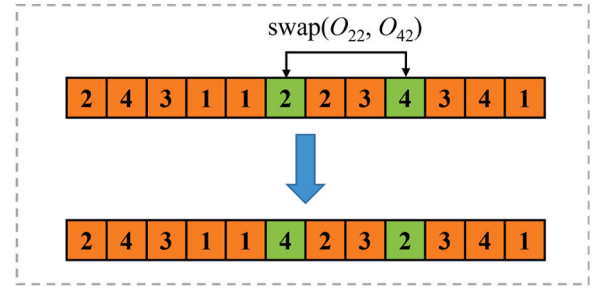
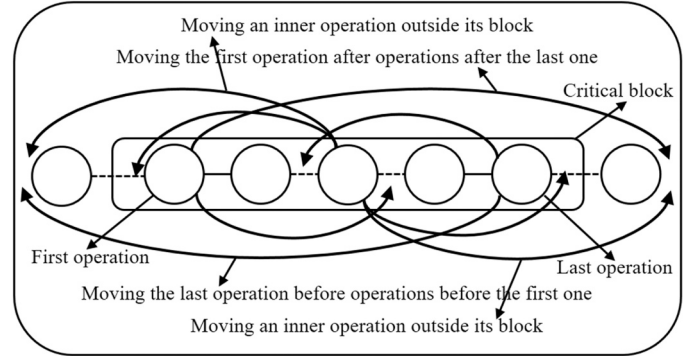


Fig. 9. The schematic example of operation mutation.

Fig. 10. N_8 neighborhood structure.

3.3. Decoding method

In the decoding process, all jobs are assigned to appropriate factories according to FA first. Then, each operation of jobs is scheduled to suitable machines according to MS. After assigning the processing factory and machine for each operation, the DFJSP degenerates into a traditional JSP. Finally, the processing sequence of operations on machines should be determined according to OS. There are three common decoding methods including active, semi-active, and hybrid decoding for JSP. This paper adopts the active decoding method to transform OS into a feasible schedule. The steps of our decoding method are described as follows:

Step 1: Assign jobs to suitable factories according to FA.

Step 2: Schedule each operation of jobs on machines according to MS.

Step 3: Decode OS into its corresponding operation sequence.

Step 4: Determine the processing sequence of operations on machines, and calculate the start time and the finish time of each operation.

Step 4.1: Assume operation O_{ij} be processed on machine M_{ij} and its processing time is pt_{ij} , and let T_x be the end time of the last operation on M_{ij} . Since precedence constraints among operations of the same job, operation O_{ij} could only be started when its last operation O_{ij-1} has been finished. Let T_y be the end time of O_{ij-1} . The allowable start time T_s of operation O_{ij} can be calculated by $T_s = \max\{T_x, T_y\}$.

Step 4.2: Find all time intervals on M_{ij} . Check each time interval $[S_o, E_o]$ in turn: if $\max\{T_s, S_o\} + pt_{ij} \leq E_o$, the earliest starting time of O_{ij} is $S_{ij} = \max\{T_s, S_o\}$, else check the next time interval. If there is no time interval satisfying the condition, operation O_{ij} is scheduled to the end of the last operation on M_{ij} . The earliest starting time is $S_{ij} = T_x$, and the finish time $C_{ij} = S_{ij} + pt_{ij}$.

Step 4.3: Go to Step 4.1 until the start and finish time of each operation have been calculated.

Step 5: Calculate the global makespan according to Eq. (6).

To describe the decoding method in more detail, we decode the encoding FA, MS, and OS in Fig. 1. The Gantt charts of F_1 and F_2 are shown in Figs. 2–3. The makespan of F_1 is 8, and the makespan of F_2 is 4.

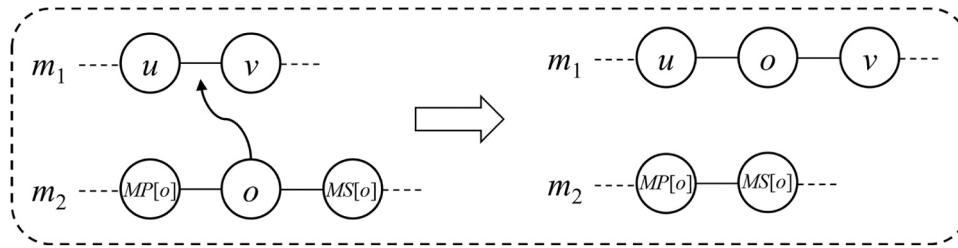


Fig. 11. N_k neighborhood structure.

Table 2

Levels of the parameters.

Parameters	Level			
	1	2	3	4
PS	10	20	30	40
P_c	0.6	0.7	0.8	0.9
P_m	0.05	0.1	0.2	0.3
$maxIter$	1000	2000	3000	4000

Table 3

Orthogonal array of the parameters.

Experiment number	Parameter				AVG
	PS	P_c	P_m	$maxIter$	
1	10	0.6	0.05	1000	410.2
2	10	0.7	0.1	2000	407.8
3	10	0.8	0.2	3000	401.6
4	10	0.9	0.3	4000	404.6
5	20	0.6	0.1	3000	404.1
6	20	0.7	0.05	4000	402.5
7	20	0.8	0.3	1000	402.3
8	20	0.9	0.2	2000	399.9
9	30	0.6	0.2	4000	399.5
10	30	0.7	0.3	3000	400.9
11	30	0.8	0.05	2000	399.4
12	30	0.9	0.1	1000	400.9
13	40	0.6	0.3	2000	399.0
14	40	0.7	0.2	1000	402.1
15	40	0.8	0.1	4000	398.5
16	40	0.9	0.05	3000	400.7

Thus, we can calculate that the global makespan of all factories is 8.

3.4. Crossover operator

This section designs three crossover operations to generate feasible offspring individuals. The first crossover operation is factory crossover, the second one is machine crossover, and the last one is operation crossover. In the crossover procedure, the three crossover operations are randomly selected to be performed. The factory with the maximum completion time is defined as the critical factory. The three crossover operators are designed based on the critical factory to reduce the global makespan.

- (1) Factory crossover. Identify the critical factories of father and mother. Randomly select two crossover points from FA. The elements between the two points in FA must contain the critical factory of father or mother. Append the elements between the two points in father to offspring 1, and append the elements outside the two points in mother to offspring 1. Similarly, append the elements between the two points in mother to offspring 2, and append the elements outside the two points in father to offspring 2. Fig. 4 presents a schematic example of factory crossover. We assume that F_2 is the critical factory. Obviously, the elements between the crossover points in father contain F_2 .
- (2) Machine crossover. Randomly select two crossover points on the vertical axis in MS. The jobs in [Point 1, Point 2] must contain at least one job in the critical factory of father or mother. Append the elements between the two points in father to offspring 1, and



Fig. 12. The trend of the factor level.

Table 4

Computational results of HG TSA, GA, and TS for LA01, 06, 11, and 16 instances.

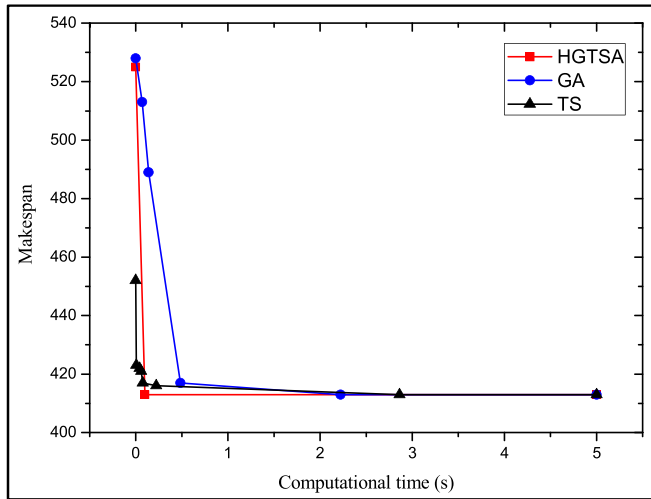
Problem	UB (LB)	HG TSA			GA			TS		
		Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)
LA01	413 (413)	413	413.0	0.1	413	415.0	0.3	413	414.8	0.5
LA06	424 (413)	413	416.2	3.13	416	419.1	7.4	414	418.2	16.3
LA11	541 (413)	536	537.0	6.5	540	541.0	9.0	539	539.8	9.3
LA16	717 (717)	717	717.0	0.1	717	718.8	3.1	717	718.0	3.6

The best solutions of HG TSA better than those of compared algorithms are indicated in bold.

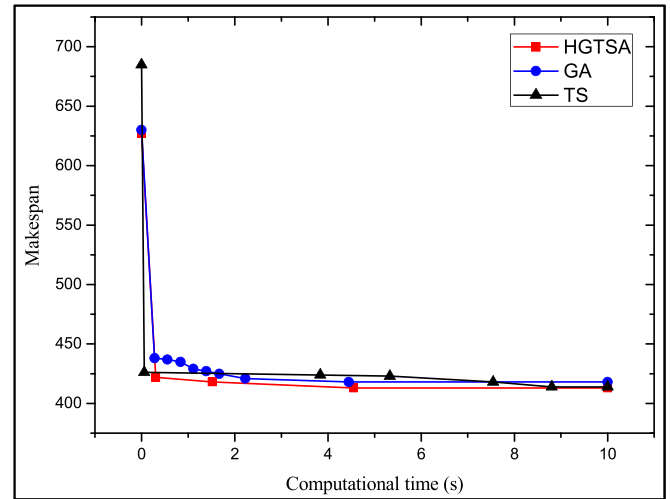
append the elements outside the two points in mother to offspring 1. Similarly, append the elements between the two points in mother to offspring 2, and append the elements outside the two points in father to offspring 2. Fig. 5 gives a schematic example of machine crossover. The jobs in [Point 1, Point 2] are I_2 and I_3 . According to FA of father shown in Fig. 4, I_2 , I_3 , and I_4 are in the critical factory F_2 . Hence, the jobs in [Point 1, Point 2] contain two jobs in the critical factory of father.

- (3) Operation crossover. Pick out all jobs in the critical factory of father or mother. Randomly divide these jobs into two sub-job sets $JobSet_1$ and $JobSet_2$. Append the elements which belong to $JobSet_1$ in father to the same position in offspring 1, and append

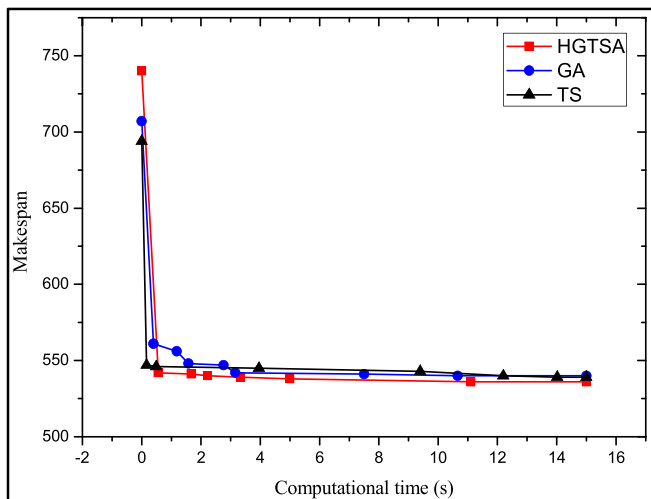
the elements which belong to $JobSet_2$ in mother to the same position in offspring 1. Similarly, append the elements which belong to $JobSet_2$ in father to the same position in offspring 2, and append the elements which belong to $JobSet_2$ in mother to the same position in offspring 2. The elements which do not belong to $JobSet_1$ or $JobSet_2$ remain in their original positions. Fig. 6 provides a schematic example of operation crossover. The job set in the critical factory of father is $\{I_2, I_3, I_4\}$, and the two sub-job sets are $JobSet_1 = \{I_2\}$ and $JobSet_2 = \{I_3, I_4\}$. I_1 is not in the critical factory F_2 , so the element positions of I_1 are not moved.



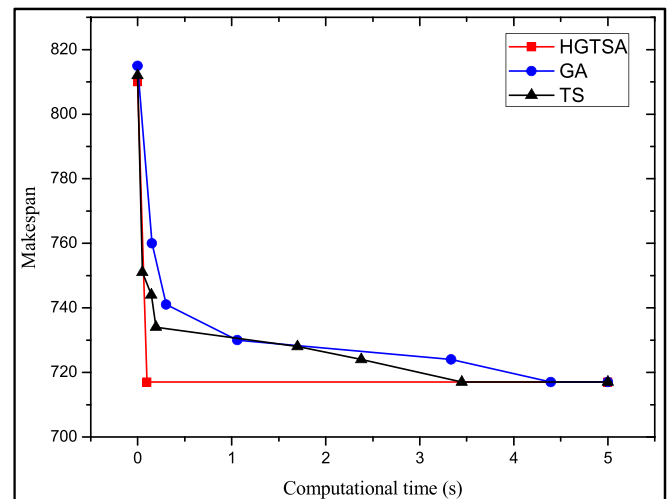
(a) LA01



(b) LA06

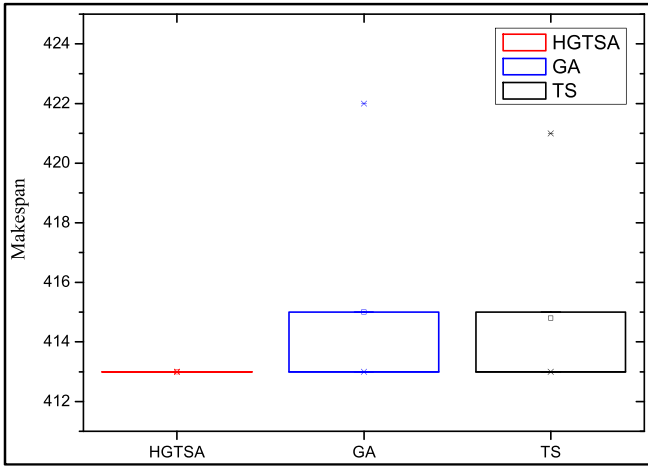


(c) LA11

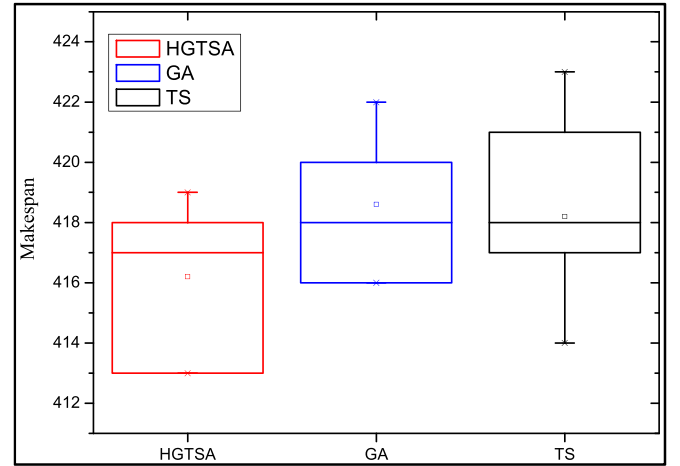


(d) LA16

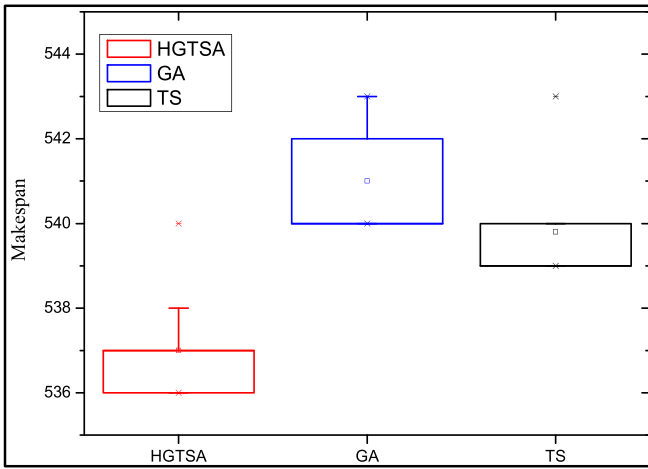
Fig. 13. Convergence curves of HG TSA, GA, and TS for LA01, 06, 11, and 16 instances.



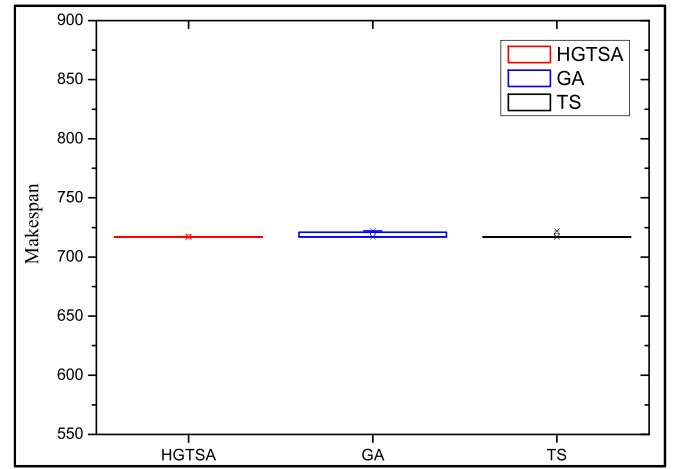
(b) LA01



(b) LA06



(d) LA11



(d) LA16

Fig. 14. Box plots of best fitness for LA01, 06, 11, and 16 instances.

3.5. Mutation operator

Similar to the crossover operations, three mutation operations are also designed based on the critical factory. The first mutation operation is factory mutation, the second one is machine mutation, and the last one is operation mutation. In the mutation procedure, the three mutation operations are randomly selected to be performed. The mutation operations are described as follows:

- (1) Factory mutation. Randomly select a job in the critical factory and move it to other factories in FA. The makespan of the last finished factory will be reduced in this way, so the global makespan of all factories may be reduced. Fig. 7 shows a schematic example of factory mutation. We assume that F_2 is the critical factory. The original factory of I_2 is F_2 . After performing factory mutation, I_2 is moved to F_1 .
- (2) Machine mutation. Pick out all jobs in the critical factory. For each picked job, randomly select an operation to change its processing machine in MS. Fig. 8 presents a schematic example of machine mutation. The jobs in the critical factory F_2 are $\{I_2, I_3, I_4\}$. The processing machine m_1 of O_{23} is changed to m_3 , the processing machine m_1 of O_{31} is changed to m_2 , and the processing machine m_1 of O_{32} is changed to m_2 .

- (3) Operation mutation. Randomly select two operations belonging to different jobs in the critical factory, and then swap their positions in OS. Since the processing sequence of operations in the critical factory is changed, the global makespan of all factories may be reduced. Fig. 9 illustrates a schematic example of operation mutation. The jobs in the critical factory F_2 are $\{I_2, I_3, I_4\}$, so the operations of I_2, I_3 , and I_4 can be selected to swap their positions. In Fig. 9, Operations O_{22} and O_{42} are selected to swap their positions.

3.6. Tabu search algorithm

3.6.1. The procedure of TS

Tabu search (TS) is a famous meta-heuristic algorithm [29], having been successfully used for a variety of scheduling problems. HGTTA performs TS on the critical factory and not on other factories. This can effectively reduce the global makespan of all factories without spending too much computation time. The characteristic of TS is to employ a tabu list to avoid re-visiting the solutions which have been searched previously. If a solution is prohibited by the tabu list, it cannot be selected as a new solution for the next generation. TS is composed of several elements, including neighborhood structures, move attributes, the tabu list, the aspiration criterion, and the termination criterion. This paper incorporates TS into the proposed HGTTA to improve its local search

ability. The neighborhood structure is the most critical element in TS, it will be introduced in detail in the next section. The remaining elements of TS can be referred to by Zhang et al. [30]. The pseudo-code of TS is shown in Algorithm 2.

Algorithm 2. The procedure of TS.

```

1: Input: an initial solution  $x_0$ 
2: Output: the best solution  $x_b$  found so far
3:  $x_b \leftarrow x_0$ 
4:  $T \leftarrow \emptyset$  //  $T$  is the tabu list
5:  $N \leftarrow 0, N_{\max} \leftarrow \text{improveIter}$ 
6: While ( $x_b$  reaches the lower bound or the total iteration number achieves a maximum value) do
7:    $Children \leftarrow \emptyset$  //  $Children$  is the set of neighborhood solutions of  $x_0$ 
8:   While ( $|Children| < M$ ) do //  $M$  is the size of neighborhood solutions
9:     generate a neighborhood solution  $x'$  of  $x_0$ 
10:     $Children \leftarrow Children \cup x'$ 
11:   End while
12:    $x' \leftarrow \arg \min (f(x) : x \in Children \wedge (x \text{ is not tabu} \vee x \text{ satisfies aspiration criterion}))$ 
13:   If ( $f(x') < f(x_b)$ ) then
14:      $x_b \leftarrow x', N \leftarrow 0$ 
15:   Else
16:      $N \leftarrow N + 1$ 
17:   End if
18:   If  $N == N_{\max}$  then
19:     Randomly select a solution  $x''$  from  $Children$ 
20:      $x_0 \leftarrow x'', N \leftarrow 0$ 
21:   Else
22:      $x_0 \leftarrow x'$ 
23:   End if
24:   Update  $T$ 
25: End while
26: Return the best solution  $x_b$ 

```

3.6.2. Neighborhood structure

The critical path is the longest in a scheduling scheme. For FJSP, if a movement cannot change the critical path of the original scheduling solution, the makespan of the new solution cannot be reduced. Hence, this paper introduces two neighborhood structures, N_8 [31] and N_k [32], to generate neighborhood solutions. The characteristic of these two neighborhood structures is that they both change the critical path by moving the critical operations.

Compared with the previously proposed neighborhood structures [30,33,34], N_8 considers not only the movement of critical operations within a critical block but also the movement of critical operations outside the critical block. Hence, N_8 can search a more extensive solution space than the previous neighborhood structures, thereby increasing the probability of TS obtaining the optimal solution. The

schematic diagram of N_8 is shown in Fig. 10.

The movement of N_k is to schedule a critical operation to another available machine. A schematic diagram of N_k is shown in Fig. 11. A critical operation o is scheduled from m_2 to m_1 . $MP[o]$ and $MS[o]$ denote the machine predecessor and successor of o on m_1 . u and v denote two operations on m_2 . After performing the movement of N_k , the critical

operation o is scheduled to m_1 . The new position of o is between u and v . Because critical operation o leaves its previous critical block, the critical path of the original scheduling scheme is changed.

4. Experimental studies and results

In this section, the parameters of HGTSA are adjusted first. Then, HGTSA is compared with GA and TS to verify the effectiveness of the hybrid strategy. Finally, 69 famous benchmark instances proposed by Giovanni and Pezzella [21] are used to test the performance of HGTSA. Although the benchmark was initially designed for the traditional JSP, it has been widely used to evaluate the optimization performance of algorithms for solving DFJSP in recent years. So far, there are still 13 instances out of these instances unresolved. The HGTSA was implemented

Table 5

Computational results for the DFJSP with 2 factories.

Problem	Size	UB (LB)	HGTSA			IGA			GA_JS			GA_OP			CRO
			Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	
LA01	10 × 5	413 (413)	413	413	0.13	413	413	12.0	413	413	7.1	413	413	4.7	413
LA02	10 × 5	394 (394)	394	394	0.44	394	394	11.2	394	394	6.6	394	394	4.7	394
LA03	10 × 5	349 (349)	349	349	0.52	349	349	10.8	349	349	7.0	349	349	5.2	349
LA04	10 × 5	369 (369)	369	369	0.53	369	369	11.4	369	369	6.4	369	369	5.0	369
LA05	10 × 5	380 (380)	380	380	0.38	380	380	8.0	380	380	6.6	380	380	5.0	380
LA06	15 × 5	424 (413)	413*	416.2	3.13	445	449.6	45.8	424	435.8	25.8	424	432.7	23.9	424
LA07	15 × 5	390 (376)	386*	389.1	2.58	412	419.2	50.2	398	408.5	26.3	390	403.6	24.6	398
LA08	15 × 5	397 (369)	391*	396.3	2.75	420	427.8	53.8	406	417.4	26.4	397	411.7	23.7	406
LA09	15 × 5	444 (382)	436*	441.2	2.99	469	474.6	45.2	447	459	27.0	444	455.7	24.1	463
LA10	15 × 5	443 (443)	443	443	0.81	445	448.6	45.0	443	444.1	24.9	443	443.2	21.9	445
LA11	20 × 5	541 (413)	536*	537	6.50	570	571.6	126.0	548	557.1	64.6	541	549.9	62.5	553
LA12	20 × 5	474 (408)	468*	469.6	6.81	504	508	116.0	480	492.5	65.5	474	482.3	63.9	500
LA13	20 × 5	529 (382)	519*	520.6	5.14	542	552.2	125.4	533	538.4	66.1	529	538.1	62.3	551
LA14	20 × 5	542 (443)	535*	536.4	4.51	570	576	122.2	542	557.3	63.3	544	553.7	63.0	581
LA15	20 × 5	554 (378)	545*	547.1	4.92	584	588.8	119.6	562	568.7	63.6	554	566.6	63.1	597
LA16	10 × 10	717 (717)	717	717	0.14	717	717	140.2	717	717	53.3	717	717	47.8	717
LA17	10 × 10	646 (646)	646	646	0.41	646	646	112.6	646	646	52.0	646	646	48.7	646
LA18	10 × 10	663 (663)	663	663	0.68	663	663	132.4	663	663	53.9	663	663	48.7	663
LA19	10 × 10	617 (617)	617	617	2.39	617	617.2	147.2	617	622.1	58.4	617	617	57.9	617
LA20	10 × 10	756 (756)	756	756	0.68	756	756	99.8	756	756	53.9	756	756	48.3	756
MT06	6 × 6	47 (47)	47	47	0.05	47	47	2.0	47	47	3.4	47	47	2.4	-
MT10	10 × 10	655 (655)	655	655	0.76	655	655	173.0	655	655	54.0	655	655	49.6	-
MT20	20 × 5	525 (387)	513*	515.8	4.89	560	566	121.2	529	547.7	59.5	525	534.4	61.4	-

The best makespans of HGTSA better than those of comparison algorithms are indicated in bold.

Newly found upper bounds by HGTSA are indicated by *.

“-” indicates the corresponding data is not available.

in Java and performed on a computer with an Intel(R) Core i9–9900 K CPU 3.6 GHz and 16.0 GB RAM. The code can be found by the website:

<https://github.com/54gl/A-Hybrid-Genetic-Tabu-Search-Algorithm.git>.

All the instances were tested 10 times independently and the best solutions were recorded. The unit of computational time given in the following tables is second, and the symbol “-” indicates the data unavailable.

4.1. Parameter settings

Before testing the optimization performance of HGTSA, this section should adjust the parameters of the algorithm first. The HGTSA contains four parameters, population size PS , crossover probability P_c , mutation probability P_m , and maximum iterations of tabu search $maxIter$. The larger the value of PS , the more individuals in the population, and the larger the solution space to be explored, but the computational time consumed by HGTSA also increases. The appropriate P_c and P_m can

Table 6

Computational results for the DFJSP with 3 factories.

Problem	Size	UB (LB)	HGTSA			IGA			GA_JS			GA_OP			CRO
			Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	
LA01	10 × 5	413 (413)	413	413	0.11	413	413	4.6	413	413	3.7	413	413	3.0	413
LA02	10 × 5	394 (394)	394	394	0.36	394	394	3.6	394	394	3.8	394	394	3.0	394
LA03	10 × 5	349 (349)	349	349	0.44	349	349	3.8	349	349	3.8	349	349	3.1	349
LA04	10 × 5	369 (369)	369	369	0.41	369	369	3.8	369	369	3.8	369	369	3.0	369
LA05	10 × 5	380 (380)	380	380	0.04	380	380	2.6	380	380	3.7	380	380	2.9	380
LA06	15 × 5	413 (413)	413	413	0.71	413	413	17.4	413	413	8.6	413	413	7.9	413
LA07	15 × 5	376 (376)	376	376	0.70	376	376	18.2	376	376	9.9	376	376	9.1	381
LA08	15 × 5	369 (369)	369	369	0.66	369	369	19.6	369	369	9.1	369	369	8.7	403
LA09	15 × 5	382 (382)	382	382	0.67	382	387.4	17.8	382	382	10.2	382	382	11.0	437
LA10	15 × 5	443 (443)	443	443	0.64	443	443	17.0	443	443	9.2	443	443	8.4	443
LA11	20 × 5	413 (413)	413	413	1.00	425	436.8	50.6	413	418	27.3	413	418	28.9	590
LA12	20 × 5	408 (408)	408	408	1.08	408	408	44.6	408	408	19.3	408	408	22.1	483
LA13	20 × 5	395 (382)	382*	388.1	3.59	419	430.2	45.8	398	409.5	26.7	395	408.4	29.6	419
LA14	20 × 5	443 (443)	443	443	1.02	443	448.8	48.8	443	443	20.3	443	443	23.1	470
LA15	20 × 5	417 (378)	395*	406.1	4.74	451	456	42.2	420	426.8	28.0	417	430	30.5	451
LA16	10 × 10	717 (717)	717	717	0.57	717	717	36.0	717	717	21.8	717	717	21.1	717
LA17	10 × 10	646 (646)	646	646	0.49	646	646	31.6	646	646	21.2	646	646	21.1	646
LA18	10 × 10	663 (663)	663	663	0.60	663	663	36.8	663	663	21.9	663	663	20.6	663
LA19	10 × 10	617 (617)	617	617	0.72	617	617	62.4	617	617	21.3	617	617	20.2	617
LA20	10 × 10	756 (756)	756	756	0.05	756	756	34.2	756	756	21.1	756	756	20.8	756
MT06	6 × 6	47 (47)	47	47	0.06	47	47	1.0	47	47	2.4	47	47	1.8	-
MT10	10 × 10	655 (655)	655	655	0.66	655	655	50.	655	655	22.4	655	655	21.8	-
MT20	20 × 5	397 (387)	387*	392.7	5.01	439	442.6	48.2	408	418.5	26.8	397	412.7	29.5	-

The best makespans of HGTSA better than those of comparison algorithms are indicated in bold.

Newly found upper bounds by HGTSA are indicated by *.

“-” indicates the corresponding data is not available.

Table 7
Computational results for the DFJSP with 4 factories.

Problem	Size	UB (LB)	HGTSA			IGA			GA_JS			GA_OP			CRO
			Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	Best	M_{av}	T_{av} (s)	
LA01	10 × 5	413 (413)	413	413	0.19	413	413	1.8	413	413	3.8	413	413	2.4	413
LA02	10 × 5	394 (394)	394	394	0.36	394	394	1.8	394	394	3.7	394	394	2.4	394
LA03	10 × 5	349 (349)	349	349	0.03	349	349	2.2	349	349	3.7	349	349	2.4	349
LA04	10 × 5	369 (369)	369	369	0.37	369	369	2.0	369	369	3.7	369	369	2.4	369
LA05	10 × 5	380 (380)	380	380	0.01	380	380	1.0	380	380	3.6	380	380	2.4	380
LA06	15 × 5	413 (413)	413	413	0.63	413	413	9.0	413	413	6.6	413	413	5.5	413
LA07	15 × 5	376 (376)	376	376	0.68	376	376	9.6	376	376	7.3	376	376	5.7	376
LA08	15 × 5	369 (369)	369	369	0.57	369	369	12.6	369	369	6.7	369	369	5.7	387
LA09	15 × 5	382 (382)	382	382	0.61	382	382	11.6	382	382	7.3	382	382	5.7	409
LA10	15 × 5	443 (443)	443	443	0.66	443	443	7.8	443	443	6.6	443	443	5.4	443
LA11	20 × 5	413 (413)	413	413	0.95	413	413	29.6	413	413	13.7	413	413	13.4	413
LA12	20 × 5	408 (408)	408	408	0.77	408	408	26.6	408	408	13.4	408	408	11.9	424
LA13	20 × 5	382 (382)	382	382	0.83	382	382	27.6	382	382	15.9	382	382	15.6	394
LA14	20 × 5	443 (443)	443	443	0.88	443	443	29.8	443	443	12.9	443	443	12.2	463
LA15	20 × 5	378 (378)	378	378	1.86	397	402	28.8	378	381.6	18.5	378	385.8	17.9	407
LA16	10 × 10	717 (717)	717	717	0.01	717	717	20.2	717	717	14.6	717	717	12.4	717
LA17	10 × 10	646 (646)	646	646	0.01	646	646	16.4	646	646	13.4	646	646	12.2	646
LA18	10 × 10	663 (663)	663	663	0.52	663	663	24.4	663	663	14.8	663	663	12.6	663
LA19	10 × 10	617 (617)	617	617	0.69	617	617	33.0	617	617	13.3	617	617	11.3	617
LA20	10 × 10	756 (756)	756	756	0.01	756	756	18.0	756	756	13.9	756	756	11.8	756
MT06	6 × 6	47 (47)	47	47	0.01	47	47	0.2	47	47	2.0	47	47	1.7	-
MT10	10 × 10	655 (655)	655	655	0.59	655	655	31.2	655	655	14.4	655	655	13.3	-
MT20	20 × 5	387 (387)	387	387	0.88	387	388.4	27.0	387	387	14.5	387	387	15.7	-

“–” indicates the corresponding data is not available.

balance the exploration and the exploitation well, and avoid the hybrid algorithm from getting stuck in the local optimal solution. The larger the value of *maxIter*, the stronger the local search ability of TS. However, too much exploration of the local space will also make HGTSA easy to get stuck in the local optima. An instance LA08 [21] with 2 factories, 15 jobs, and 5 machines is applied to the parameter setting by the Taguchi method of design of experiment (DOE). The levels of each parameter are listed in Table 2. An orthogonal array L_{16} shown in Table 3 is used to determine the parameter combination. The trend graph of these four parameters is shown in Fig. 12. As a result from the experimental analysis, we can get the best parameter setting: $PS = 40$, $P_c = 0.8$, $P_m = 0.2$, and $maxIter = 4000$.

4.2. Evaluation of the hybrid strategy

In this subsection, four instances with different scales $m \times n \times f$ are selected to test the effectiveness of the hybrid strategies. m denotes the job number, n denotes the machine number, and f denotes the factory number. The four instances are LA01, LA06, LA11, and LA16, and their problem scales are $10 \times 5 \times 2$, $15 \times 5 \times 2$, $20 \times 5 \times 2$, and $10 \times 10 \times 2$. We compare HGTSA with GA and TS for the four instances. After ten independent runs, the comparison results of HGTSA, GA, and TS are shown in Table 4. LB denotes the lower bound and UB denotes the upper bound. Best, M_{av} , and T_{av} represent the best makespan, the average makespan, and the average computational time of algorithms, respectively. The convergence curves of the three algorithms are shown in Fig. 13, where the X axis represents the computational time (unit: seconds) of each algorithm, and the Y axis represents the makespan of each algorithm obtained in the evolution process. Besides, the box plot of best fitness values obtained by HGTSA, GA, and TS in ten runs is shown in Fig. 14.

Table 4 shows that all algorithms obtain the optimal solutions LA01–413 and LA16–717, but only HGTSA achieves the optimal solutions LA06–413 and LA11–536. The values of M_{av} obtained by HGTSA are lower than those of GA and TS. The average makespan shows that HGTSA is more robust than GA and TS. The values of T_{av} obtained by HGTSA are better than those of GA and TS. The average computational time indicates that HGTSA is more efficient than GA and TS. Fig. 13 presents that the convergence speed of HGTSA is faster than that of the

comparison algorithms. Moreover, compared with GA and TS, HGTSA is easier to jump out of the local optimum. Fig. 14 provides the box plot of best fitness obtained by HGTSA, GA, and TS in ten runs. We can clearly observe that the result distribution of HGTSA is more concentrated than that of GA and TS. The above experimental analysis proves that HGTSA outperforms GA and TS in terms of solution quality and computation efficiency. Hence, our hybrid strategy is effective in solving DFJSP.

4.3. Comparisons of HGTSA with other algorithms

In this subsection, 69 benchmark instances proposed by Giovanni and Pezzella [21] are used to test the performance of HGTSA. Although the benchmark has been proposed for more than ten years, there are still 13 instances unresolved. Four state-of-the-art algorithms, IGA [21], GA_JS [35], GA_OP [36], and CRO [23], are used to compare with HGTSA. All of them are the most effective algorithms for solving DFJSP. IGA was the first algorithm proposed to solve the DFJSP benchmark. CRO has reached the lower bounds for 36 instances. GA_JS and GA_OP are currently the most effective methods for DFJSP. GA_JS and GA_OP have reached the lower bounds for 56 instances.

Tables 5, 6, and 7 present the comparison results of HGTSA and other algorithms for DFJSP with 2, 3, and 4 factories, respectively. IGA, GA_JS, and GA_OP provide the best makespan, the average makespan, and the average computational time. CRO gives only the best makespan because other data is not available. From Table 5, it can be observed that HGTSA finds new upper bounds for 10 out of 23 instances. The values of Best obtained by HGTSA are obviously lower than those obtained by IGA, GA_JS, and GA_OP. The values of M_{av} obtained by HGTSA are lower than those obtained by IGA, GA_JS, GA_OP, and CRO. The computational results verify that HGTSA outperforms these comparison algorithms in terms of solution quality. Besides, the values of T_{av} obtained by HGTSA are better than those obtained by IGA, GA_JS, and GA_OP. The computational time shows that HGTSA is more efficient than IGA, GA_JS, and GA_OP in solving DFJSP. From Table 6, we can observe that HGTSA reaches the lower bounds for all instances except LA15. Moreover, HGTSA finds three new solutions LA13–382, LA15–395, and MT20–387, of which the hybrid algorithm reaches the lower bounds for LA13 and MT20. The values of M_{av} are equal to the values of Best obtained by HGTSA for most instances. The computational results show that HGTSA

has strong robustness. In multiple runs, the HGTSA can stably obtain the optimal solution for most instances. From Table 7, it is clear that HGTSA reaches the lower bounds for all instances. IGA cannot reach the lower bounds for LA15, and CRO cannot reach the lower bounds for LA8–9, 12–15. Although GA_JS and GA_OP also reach the lower bounds for all instances, HGTSA can achieve the values of M_{av} equal to the values of Best, and GA_JS and GA_OP cannot.

In sum, HGTSA improves the upper bounds for 13 out of 69 instances. Three of these 13 new upper bounds, LA06–413 with 2 factories, LA13–382 with 3 factories, and MT20–387 with 3 factories, reach the lower bounds. The experimental results demonstrate that HGTSA performs better than the comparison algorithms in terms of both the solution quality and the computational efficiency.

4.4. Discussions

In the experimental section, three types of experiments are conducted in this paper. Firstly, the orthogonal experiments of the algorithm parameters are carried out for the proposed algorithm. On this basis, reasonable algorithm parameters are set. Then, this paper compares the performance of the hybrid algorithm with that of the single algorithm. The experimental results show that the hybrid algorithm has obvious advantages in both computational efficiency and computational results. Finally, the proposed algorithm is compared with the best algorithm in the existing literature. The experimental results show that the proposed algorithm has obvious advantages in different sizes of instances.

We discuss and analyze the third experimental result in more detail. The experimental results in this part demonstrate that HGTSA can be successfully applied to DFJSP. Compared with four state-of-the-art algorithms including IGA, GA_JS, GA_OP, and CRO, HGTSA achieves better makespans and spends less computational time for most instances. It can be concluded that HGTSA is more effective and efficient in solving JSP than IGA, GA_JS, GA_OP, and CRO. In particular, HGTSA has found 13 new solutions. The advantages of HGTSA can be summarized as follows. Firstly, HGTSA combines GA and TS to balance its exploitation and exploration well. The evaluation of the hybrid strategy shows that HGTSA is more effective in solving DFJSP than GA and TS. Secondly, the introduction of two effective neighborhood structures (i.e., N_g and N_k) into HGTSA greatly increases its local search ability. Thirdly, three new crossover operators and mutation operators are designed to generate feasible offspring individuals. The crossover operators guarantee the discreteness of the population, and the mutation operators ensure individuals do not fall into the local optimum. Finally, we choose appropriate parameters to maximize the performance of HGTSA while saving computational time.

Combined with the above characteristics, the algorithm proposed in this paper can search the solution space of the problem sufficiently, and HGTSA outperforms the comparison algorithms in both solution quality and computational time. However, the more essential reason for the effectiveness of the algorithm is that it makes full use of the domain knowledge of the problem, that is, the characteristics of the problem. Among them, the most notable one is the neighborhood structure used in this paper, they can not only effectively avoid infeasible solutions in FJSP, but also minimize the number of neighborhood solutions which do not improve the current solution.

Although this paper proposes a hybrid algorithm for distributed flexible job shop scheduling problem, the algorithm can still be used for job shop scheduling problem and flexible job shop scheduling problem, which makes the algorithm proposed in this paper have a wider application scenario. This feature is of great significance to some software development enterprises. For example, an enterprise needs to develop shop scheduling algorithms for different shop scheduling scenarios. When using the algorithm proposed in this paper, the efficiency of software development can be greatly improved and the development cost can be reduced.

5. Conclusions and future works

This paper proposes a hybrid genetic tabu search algorithm (HGTSA) combining with the genetic algorithm (GA) and tabu search (TS) algorithm for solving the distributed and flexible job shop scheduling problem (DFJSP). 69 famous benchmark instances have been selected to test the performance of HGTSA. The computational results demonstrate that HGTSA performs better than four state-of-the-art algorithms in terms of the solution quality and the computational efficiency. In particular, HGTSA has found 13 new solutions.

The contributions of our work can be concluded as follows: 1) A hybrid algorithm is designed for solving DFJSP. The hybrid algorithm combines the GA and TS to balance its exploration and exploitation well. The computational results demonstrate that HGTSA outperforms the previously proposed algorithms; 2) Three new crossover operators and mutation operators are devised based on the critical path. The crossover operators make the population discrete, and the mutation operators avoid individuals falling into the local optimum; 3) HGTSA has found 13 newly upper bounds out of 69 benchmark instances.

Although it is pointed out that the reason for the effectiveness of the algorithm is that it makes full use of the domain knowledge of the problem, except for the two domain structures of the paper, the other domain knowledge in the algorithm is very vague and does not necessarily have theoretical guarantees. Therefore, a more in-depth analysis of the neighborhood knowledge in the problem is needed in future work. In addition, possible future research directions are as follows: 1) This paper studies a homogeneously distributed workshop. The type and quantity of machines in each workshop are the same. However, in an actual production environment, the type and quantity of machines in each workshop may vary. Therefore, we can further study the heterogeneously distributed workshop. 2) This paper considers only one criterion, the maximum completion time. We can also consider other scheduling criteria such as machine utilization, flow time, and total flow time; 3) We can apply the HGTSA to other shop scheduling problems.

Funding

This work was supported by the National Natural Science Foundation of China under Grant Nos. 51825502, U21B2029, and 52188102.

CRediT authorship contribution statement

Jin Xie: Conceptualization, Methodology, Writing – original draft. **Xinyu Li:** Supervision, Writing – review & editing. **Liang Gao:** Supervision, Project administration. **Lin Gui:** Methodology and Editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Kim HJ, Lee JH. Look-ahead based reinforcement learning for robotic flow shop scheduling. *J Manuf Syst* 2023;68:160–75.
- [2] Zhao F, Di S, Wang L. A hyperheuristic with Q-learning for the multiobjective energy-efficient distributed blocking flow shop scheduling problem. *IEEE Trans Cyber* 2023;53(5):3337–50.
- [3] Han Y, Li J, Sang H, Liu Y, Gao K, Pan Q. Discrete evolutionary multi-objective optimization for energy-efficient blocking flow shop scheduling with setup time. *Appl Soft Comput* 2020;93:106343.
- [4] Sanogo K, Benhafsa AM, Sahnoun MH, Bettayeb B, Abderrahim M, Bekrar A. A multi-agent system simulation based approach for collision avoidance in integrated job-shop scheduling problem with transportation tasks. *J Manuf Syst* 2023;68:209–26.
- [5] Gui L, Li X, Gao L, Wang C. Necessary and sufficient conditions for feasible neighbourhood solutions in the local search of the job-shop scheduling problem. *Chin J Mech Eng* 2023;36(1):1–16.

- [6] Destouet C, Tlahig H, Bettayeb B, Mazari B. Flexible job shop scheduling problem under Industry 5.0: a survey on human reintegration, environmental consideration and resilience improvement. *J Manuf Syst* 2023;67:155–73.
- [7] Li Y, Huang W, Wu R, Guo K. An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem. *Appl Soft Comput* 2020;95:106544.
- [8] Zhang G, Hu Y, Sun J, Zhang W. An improved genetic algorithm for the flexible job shop scheduling problem with multiple time constraints. *Swarm Evol Comput* 2020;54:100664.
- [9] Zhu Z, Zhou X. An efficient evolutionary grey wolf optimizer for multi-objective flexible job shop scheduling problem with hierarchical job precedence constraints. *Comput Ind Eng* 2020;14:0106280.
- [10] Gong G, Chiong R, Deng Q, Gong X. A hybrid artificial bee colony algorithm for flexible job shop scheduling with worker flexibility. *Int J Prod Res* 2020;58(14): 4406–20.
- [11] Naderi B, Azab A. Modeling and heuristics for scheduling of distributed job shops. *Expert Syst Appl* 2014;41:7754–63.
- [12] Naderi B, Ruiz R. The distributed permutation flowshop scheduling problem. *Comput Oper Res* 2010;37:754–68.
- [13] Lu C, Huang Y, Meng L, Gao L, Zhang B, Zhou J. A Pareto-based collaborative multi-objective optimization algorithm for energy-efficient scheduling of distributed permutation flow-shop with limited buffers. *Robot Comput Integr Manuf* 2022;74:102277.
- [14] Mao JY, Pan QK, Miao ZH, Gao L. An effective multi-start iterated greedy algorithm to minimize makespan for the distributed permutation flowshop scheduling problem with preventive maintenance. *Expert Syst Appl* 2021;169:114495.
- [15] Jia H, Fuh JY, Nee AY, Zhang Y. Web-based multi-functional scheduling system for a distributed manufacturing environment. *Concurr Eng* 2002;10:27–39.
- [16] Sang Y, Tan J. Intelligent factory many-objective distributed flexible job shop collaborative scheduling method. *Comput Ind Eng* 2022;164:107884.
- [17] Şahman MA. A discrete spotted hyena optimizer for solving distributed job shop scheduling problems. *Appl Soft Comput* 2021;106:107349.
- [18] Lu C, Gao L, Yi J, Li X. Energy-efficient scheduling of distributed flow shop with heterogeneous factories: a real-world case from automobile industry in China. *IEEE Trans Ind Inf* 2020;17:6687–96.
- [19] Shao Z, Shao W, Pi D. Effective heuristics and metaheuristics for the distributed fuzzy blocking flow-shop scheduling problem. *Swarm Evol Comput* 2020;59: 100747.
- [20] Shao Z, Pi D, Shao W. Hybrid enhanced discrete fruit fly optimization algorithm for scheduling blocking flow-shop in distributed environment. *Expert Syst Appl* 2020; 145:113147.
- [21] De Giovanni L, Pezzella F. An improved genetic algorithm for the distributed and flexible job-shop scheduling problem. *Eur J Oper Res* 2010;200:395–408.
- [22] Luo Q, Deng Q, Gong G, Zhang L, Han W, Li K. An efficient memetic algorithm for distributed flexible job shop scheduling problem with transfers. *Expert Syst Appl* 2020;160:113721.
- [23] Zhu K, Gong G, Peng N, Zhang L, Huang D, Luo Q, et al. Dynamic distributed flexible job-shop scheduling problem considering operation inspection. *Expert Syst Appl* 2023;224:119840.
- [24] Lu PH, Wu MC, Tan H, Peng YH, Chen CF. A genetic algorithm embedded with a concise chromosome representation for distributed and flexible job-shop scheduling problems. *J Intell Manuf* 2018;29:19–34.
- [25] Palacios JJ, González MA, Vela CR, González-Rodríguez I, Puente J. Genetic tabu search for the fuzzy flexible job shop problem. *Comput Oper Res* 2015;54:74–89.
- [26] Zhang L, Gao L, Li X. A hybrid genetic algorithm and tabu search for a multi-objective dynamic job shop scheduling problem. *Int J Prod Res* 2013;51:3516–31.
- [27] Li X, Gao L. An effective hybrid genetic algorithm and tabu search for flexible job shop scheduling problem. *Int J Prod Econ* 2016;174:93–110.
- [28] Meng L, Zhang C, Ren Y, Zhang B, Lv C. Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem. *Comput Ind Eng* 2020;142:106347.
- [29] Glover F, Laguna M. Tabu search. *Handbook of combinatorial optimization*. Boston, MA: Springer; 1998. p. 2093–229.
- [30] Zhang C, Li P, Guan Z, Rao Y. A tabu search algorithm with a new neighborhood structure for the job shop scheduling problem. *Comput Oper Res* 2007;34:3229–42.
- [31] Xie J, Li X, Gao L, Gui L. A new neighbourhood structure for job shop scheduling problems. *Int J Prod Res* 2023;61(7):2147–61.
- [32] Mastrolilli M, Gambardella LM. Effective neighbourhood functions for the flexible job shop problem. *J Sched* 2000;3:3–20.
- [33] Nowicki E, Smutnicki C. A fast taboo search algorithm for the job shop problem. *Manag Sci* 1996;42:797–813.
- [34] Balas E, Vazacopoulos A. Guided local search with shifting bottleneck for job shop scheduling. *Manag Sci* 1998;44:262–75.
- [35] Lu PH, Wu MC, Tan H, Peng YH, Chen CF. A genetic algorithm embedded with a concise chromosome representation for distributed and flexible job-shop scheduling problems. *J Intell Manuf* 2018;29:19–34.
- [36] Wu MC, Lin CS, Lin CH, Chen CF. Effects of different chromosome representations in developing genetic algorithms to solve DFJS scheduling problems. *Comput Oper Res* 2017;80:101–12.